

Technical Documentation

1. System Overview

AI Sarthi is a specialized Retrieval-Augmented Generation (RAG) system designed to serve as a healthcare assistant. It leverages a local Vector Database to perform semantic retrieval over ingested medical documents and feeds the contextual data into a fine-tuned Small Language Model (SLM) for grounded answer generation.

2. Technology Stack

- **Language:** Python
- **Web Framework:** Streamlit
- **Machine Learning / LLM:** PyTorch, PEFT (QLoRA)
- **Vector Database:** ChromaDB
- **Embeddings:** Sentence Transformers (BAAI/bge-small-en-v1.5)
- **Document Processing:** PyPDF, Pandas, LangChain Text Splitters
- **Evaluation:** NLTK (BLEU), ROUGE-Score, BERTScore
- **Environment Management:** python-dotenv

3. Repository Structure

```
healthcare-ai/
├── app/
│   ├── streamlit_app.py # Streamlit web interface and chat service wrapper
│   └── chat_service.py # Main Streamlit UI entry point
├── src/
│   ├── config/ # Interface between UI and RAG pipeline
│   ├── dataset/ # Core logic and modules
│   ├── embeddings/ # Centralized configuration (settings.py)
│   ├── evaluation/ # Dataset loading and validation logic
│   ├── ingestion/ # Embedding model instantiation
│   ├── model/ # Metric calculation (BLEU, ROUGE, BERTScore)
│   ├── rag/ # Document parsing and chunking pipeline
│   ├── retrieval/ # Inference and training (QLoRA) modules
│   └── vectorstore/ # End-to-end RAG pipeline logic
├── scripts/ # Retriever abstraction interacting with ChromaDB
│   ├── evaluate_model.py # ChromaDB client and persistence logic
│   ├── train_phi3.py # Executable entry points
│   ├── evaluation/ # CLI for running batch evaluation
│   └── ... # CLI for PyTorch QLoRA training
├── data/ # Comparison and manual eval template scripts
├── models/ # Dataset generation and chat formatting scripts
├── vector_store/ # Raw, processed, and prepared JSONL datasets
├── docs/ # Fine-tuned PyTorch checkpoints and outputs
├── reports/ # Persistent ChromaDB database files
├── plots/ # Documentation and reports
├── tests/ # Auto-generated dataset analysis reports
└── requirements.txt # Generated visualizations (Dataset stats, Training)
# Pytest test suite
# Project dependencies
```

4. Configuration

System configuration is centralized in `src/config/settings.py` and managed via a `.env` file.

- **Model Paths:** `HF_BASE_MODEL` defaults to `microsoft/Phi-3-mini-4k-instruct`. `HF_ADAPTER_PATH` points to `models/outputs`.
- **ChromaDB Config:** `CHROMA_PERSIST_DIR` (`vector_store`) and `CHROMA_COLLECTION_NAME` (`healthcare_chunks`).
- **Retrieval:** `DEFAULT_TOP_K` (15) and `DEFAULT_EMBEDDING_MODEL` (`BAAI/bge-small-en-v1.5`).
- **Ollama API:** `OLLAMA_HOST` specifies the local inference server if applicable.

5. Data Ingestion Architecture

Documents placed in `data/raw/` are processed by `src.ingestion.pipeline`. PDFs are parsed (using PyPDF) and CSVs are read (using Pandas). The text is then passed to LangChain text splitters to create smaller, semantically coherent text chunks. These chunks are saved into `data/processed/`.

6. Embedding Pipeline

The pipeline utilizes the `BAAI/bge-small-en-v1.5` model via `sentence-transformers` to generate dense vector embeddings for each processed text chunk. The embedding model is initialized in `src/embeddings/`.

7. ChromaDB Architecture

The `src/vectorstore/chromadb_store.py` module manages the local vector database.

- The store is persisted to disk at `vector_store/`.
- Document text, embeddings, and metadata (such as the source file name) are indexed into the `healthcare_chunks` collection.

8. Retrieval Module

The `Retriever` class (`src/retrieval/retriever.py`) acts as the bridge between the RAG pipeline and the vector store. It accepts a user query, delegates the query embedding process, and queries ChromaDB to fetch the `top_k` (default 15) closest matching document chunks.

9. RAG Pipeline

Located in `src/rag/rag_pipeline.py`, the `RAGPipeline` class orchestrates the complete flow:

1. Validates the incoming question.
2. Calls the `Retriever` to fetch `retrieved_chunks`.
3. Extracts unique source metadata from the chunks.
4. Uses `build_prompt()` to construct the final text.
5. Invokes `generate_response()` to pass the prompt to the fine-tuned SLM.
6. Returns a dictionary containing the question, generated answer, and a list of sources.

10. Prompt Engineering

The system utilizes a specialized prompt template built dynamically in `src/rag/prompt_builder.py`. The template embeds the `retrieved_chunks` as background context alongside the specific user query, instructing the model to rely solely on the provided context to prevent hallucinations.

11. SLM Inference

Inference logic is handled via `src/model/`.

- The base model (`microsoft/Phi-3-mini-4k-instruct`) is loaded.
- If available, the PEFT LoRA adapter weights (from `models/outputs`) are applied to the base model.
- The corresponding tokenizer formats the generation inputs, and responses are yielded back to the application layer.

12. Fine-Tuning Architecture

The fine-tuning architecture utilizes PyTorch and PEFT to train a QLoRA adapter for Phi-3.

- `scripts/train_phi3.py` serves as the entry point.
- The logic resides in `src/model/training/`.
- The synthetic dataset (`data/chat/`) is loaded, validated, and passed to a training loop.
- It optimizes adapter weights to align the model's tone and formatting with the healthcare QA dataset without altering the frozen base model parameters.

13. Evaluation Architecture

The evaluation framework (`scripts/evaluate_model.py` and `scripts/evaluation/compare_models.py`) automates batch testing.

- It reads questions from a CSV.
- Runs the RAGPipeline for each question, logging latency.
- Generates answers and compares them against references using metrics defined in `src/evaluation/metrics.py` (BLEU, ROUGE, BERTScore).
- Outputs the results into CSV files for Pandas-based aggregation and plotting.

14. Streamlit Architecture

The frontend (`app/streamlit_app.py`) leverages Streamlit to provide an interactive chatbot.

- It imports `chat_service.py` to decouple UI state from RAG execution.
- Chat history is maintained via `st.session_state`.
- RAG pipeline outputs, including cited sources, are dynamically rendered into Streamlit UI components.

15. Error Handling

The application uses custom exceptions to ensure graceful failure:

- `RAGPipelineError`: Raised for high-level pipeline failures.
- `RetrievalError`: Raised when vector store queries fail.
- `LLMError`: Raised during text generation failures.
- Streamlit catches these errors and renders user-friendly `st.error()` messages rather than crashing.

16. Logging

Python's built-in logging module is heavily utilized. Scripts such as `train_phi3.py` output detailed logs to the `logs/` directory (`train_phi3.log`, `rewrite.log`, etc.), capturing runtime behavior, warnings, and errors. The evaluation runner also configures robust console logging.

17. Testing

The repository includes a `tests/` directory configured with `pytest.ini`. The testing framework ensures the robustness of the core logic components (like ingestion and retrieval) before deployment.

18. Important Design Decisions

- **SLM (Phi-3-mini):** Selected for its excellent capability-to-size ratio, enabling deployment on local consumer hardware without cloud dependencies.
- **RAG Architecture:** Chosen over pure fine-tuning for factual queries because healthcare policies change frequently. RAG allows updating the vector store without retraining the model.
- **ChromaDB:** A lightweight, local vector store eliminates the need to deploy and manage a separate database server.
- **QLoRA:** Quantized LoRA drastically reduces the GPU memory required for fine-tuning.

19. Known Technical Limitations

- RAG latency is heavily dependent on the local machine's disk speed (for ChromaDB) and GPU VRAM (for model inference).
- The default setup utilizes semantic similarity which might miss exact keyword matches; hybrid search is not currently implemented in `chromadb_store.py`.